ICPC 板子：数学 + 计算几何

2026-10-31

Contents

数学 **1**
- 基础依赖 . . . 1
- 优先实测：基础数论与组合数 . . . 1
- 优先实测：组合数学与计数 . . . 4
- 备用：线性代数 . . . 7
- 备用：数值积分与格点计数 . . . 9

计算几何 **9**
- 基础 . . . 9
- 点和线 . . . 9
- 多边形 . . . 10
- 凸包 . . . 11
- 旋转卡壳 . . . 11
- 半平面交 . . . 12
- 闵可夫斯基和 . . . 12
- 圆 . . . 13
- 极角扫描 . . . 15
- 平面最近点对 . . . 16
- 公式 . . . 16

数学

基础依赖

```cpp
#include<bits/stdc++.h>
using namespace std;

using ll = long long;
using vi = vector<int>;
using vll = vector<ll>;
using db = double;
#define all(v) ((v).begin()),((v).end())
#define sz(v) ((int)((v).size()))
#define fi first
#define se second
#define mp make_pair
#define pb push_back

const ll MOD = 998244353; // or 1e9 + 7
const db eps = 1e-9;
```

优先实测：基础数论与组合数

快速幂

p > 0，b >= 0。mul 用 __int128 处理乘法中间结果；a、b 可以是负数。

```
ll mul(ll a, ll b, ll p) {
    a %= p;
    if(a < 0) a += p;
    b %= p;
    if(b < 0) b += p;
    return (__int128)a * b % p;
}
ll qpow(ll a, ll b, ll p = MOD) {
    ll r = 1 % p;
    a %= p;
    if(a < 0) a += p;
    while(b) {
        if(b & 1) r = mul(r, a, p);
        a = mul(a, a, p);
        b >>= 1;
    }
    return r;
}
```

阶乘、逆阶乘、组合数与 Lucas

initComb(n, p) 要求 p 是不超过 2^31-1 的素数且 0 <= n < p；C(n, k) 要求 n 不超过预处理上界。Lucas 用于小素数模数，先 initComb(p - 1, p)。不要按默认大模数预处理到 MOD - 1。

```
ll combMod;
vll fac, ifac;
void initComb(int n, ll p = MOD) {
    combMod = p;
    fac.assign(n + 1, 1);
    ifac.assign(n + 1, 1);
    for (int i = 1; i <= n; ++i) fac[i] = fac[i - 1] * i % p;
    ifac[n] = qpow(fac[n], p - 2, p);
    for (int i = n; i >= 1; --i) ifac[i - 1] = ifac[i] * i % p;
}
ll C(int n, int k) {
    if(k < 0 || k > n) return 0;
    return fac[n] * ifac[k] % combMod * ifac[n - k] % combMod;
}
ll lucas(ll n, ll k) {
    if(k == 0) return 1;
    return C(n % combMod, k % combMod) * lucas(n / combMod, k / combMod) % combMod;
}
```

线性筛

mnp 是最小质因子；每次用 mnp[x] 除掉一个质因子，可在 O(log x) 次操作内分解 x。sieve 可重复调用；n >= 1。P3383 的上界为 10^8，只测质数时删去 phi / mu 等不需要的数组，避免额外内存。

```
vi pri, mnp, phi, mu; //primes, min prime, euler phi, mobius
void sieve(int n) {
    pri.clear();
    mnp.assign(n + 1, 0);
    phi.assign(n + 1, 0);
    mu.assign(n + 1, 0);
    phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if(!mnp[i]) {
            pri.pb(i);
            mnp[i] = i;
            phi[i] = i - 1;
            mu[i] = -1;
        }
        for (int p : pri) {
```

```
            if(1LL * i * p > n) break;
            int x = i * p;
            mnp[x] = p;
            if(i % p == 0) {
                phi[x] = phi[i] * p;
                mu[x] = 0;
                break;
            }
            phi[x] = phi[i] * (p - 1);
            mu[x] = -mu[i];
        }
    }
}
```

exgcd 与逆元

exgcd：a、b 为非负整数且不同时为 0，返回 gcd(a,b)，并满足 ax + by = gcd(a,b)。inv：p >= 2，无逆元返回 -1，不要求模数为素数。

```
ll exgcd(ll a, ll b, ll& x, ll& y) {
    if(b == 0) { x = 1; y = 0; return a; }
    ll g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
ll inv(ll a, ll p) {
    a %= p;
    if(a < 0) a += p;
    ll x, y;
    if(exgcd(a, p, x, y) != 1) return -1;
    x %= p;
    if(x < 0) x += p;
    return x;
}
```

递推逆元要求 p 为不超过 2^31-1 的素数、1 <= n < p。函数名用 initInv，数组名用 iv，避免与 inv 冲突。

```
vll iv;
void initInv(int n, ll p) {
    iv.assign(n + 1, 0); iv[1] = 1;
    for (int i = 2; i <= n; ++i) iv[i] = (p - p / i) * iv[p % i] % p;
}
```

CRT / exCRT

统一用逐个合并：互质模数的普通 CRT 也可以调用这一份。输入 a[i] 是余数，m[i] 是正模数；无解返回 -1。要求两数组等长，所有模数的最小公倍数不超过 long long 范围；中间乘法用 __int128。

```
bool mergeCRT(ll& a, ll& m, ll b, ll n) { // 合并 x = a (mod m), x = b (mod n)
    b %= n;
    if(b < 0) b += n;
    ll x, y, g = exgcd(m, n, x, y);
    ll d = b - a;
    if(d % g != 0) return false;
    ll k = mul(x, d / g, n / g);
    ll mod = (ll)((__int128)m * (n / g));
    a = ((__int128)m * k + a) % mod;
    m = mod;
    return true;
}
ll excrt(vll& a, vll& m) {
    ll r = 0, mod = 1;
    for (int i = 0; i < sz(a); ++i) {
```

```
        if(!mergeCRT(r, mod, a[i], m[i])) return -1;
    }
    return r;
}
```

优先实测：组合数学与计数

组合恒等式参考 `../../模板.pdf` 第 60 页，其余补充基础模型。公式先按整数理解，取模时除法要换成逆元；使用 C 的代码都依赖前面的 initComb / C，并按 combMod 取模。其余计数代码默认按 MOD 取模。

组合恒等式与常见模型

约定 n >= 0，k < 0 或 k > n 时 $\binom{n}{k}=0$，$\binom{0}{0}=1$。以下变量均为非负整数；具体范围见公式旁说明。

模型	方案数	条件 / 解释
从 n 个不同元素中选 k 个，无序	$\binom{n}{k}$	不重复选择
从 n 个不同元素中选 k 个并排列	$n!/(n-k)!$	0 <= k <= n
多重集排列	$n!/\prod_i a_i!$	第 i 类有 a_i 个相同元素，$\sum_i a_i = n$
网格中向右走 a 步、向上走 b 步	$\binom{a+b}{a}$	无障碍、无额外路径限制

对称性与 Pascal 递推（递推式中 n >= 1）：

$$\binom{n}{k}=\binom{n}{n-k},\qquad \binom{n}{k}=\binom{n-1}{k}+\binom{n-1}{k-1}.$$

二项式定理与求和：

$$(x+y)^n=\sum_{k=0}^{n}\binom{n}{k}x^k y^{n-k},\qquad \sum_{k=0}^{n}\binom{n}{k}=2^n.$$

当 n >= 1 时，偶数项和与奇数项和都为 2^{n-1}，且 $\sum_{k=0}^{n}(-1)^k\binom{n}{k}=0$；n = 0 时偶数项和为 1，奇数项和为 0。

曲棍球棒恒等式，按选中元素中的最大值分类：

$$\sum_{i=r}^{n}\binom{i}{r}=\binom{n+1}{r+1}\quad(0\le r\le n),\qquad \sum_{i=0}^{m}\binom{n+i}{n}=\binom{n+m+1}{m}.$$

改变选择顺序，先选 m 个再从中选 k 个，等价于先选 k 个再补 m-k 个：

$$\binom{n}{m}\binom{m}{k}=\binom{n}{k}\binom{n-k}{m-k}\quad(0\le k\le m\le n).$$

Vandermonde 恒等式，从大小为 n、m 的两个不相交集合中共选 k 个：

$$\sum_{i=0}^{k}\binom{n}{i}\binom{m}{k-i}=\binom{n+m}{k}.$$

插板法（隔板法）

m 个相同物品分给 k 个不同盒子，k >= 1。盒子有区别，盒内不计顺序。

限制	方程	方案数
允许空盒	$x_1+\cdots+x_k=m,\ x_i\ge 0$	$\binom{m+k-1}{k-1}$，m >= 0

限制	方程	方案数
每盒至少一个	$x_1+\cdots+x_k=m,\ x_i\ge 1$	$\binom{m-1}{k-1}$，m >= k；否则为 0
每盒至少 l_i 个	$x_i\ge l_i$	令 $M=m-\sum_i l_i$，答案为 $\binom{M+k-1}{k-1}$；M < 0 时为 0

若每盒满足 $0\le x_i\le b$，b >= 0，用容斥排除超出上界的盒子：

$$\sum_{j=0}^{\min(k,\lfloor m/(b+1)\rfloor)}(-1)^j\binom{k}{j}\binom{m-j(b+1)+k-1}{k-1}.$$

若上界各不相同，$0\le x_i\le b_i$，枚举被指定超界的盒子集合 S，将这些变量各减去 b_i+1：

$$\sum_{S\subseteq\{1,\dots,k\}}(-1)^{|S|}\binom{m-\sum_{i\in S}(b_i+1)+k-1}{k-1}.$$

剩余物品数为负的项按 0 处理，不调用负上标的 C。各不相同的上下界先平移为 $0\le y_i\le u_i-l_i$；若某个 u_i < l_i 则无解。公式中的最大组合数上标必须在预处理范围内且小于 combMod。

容斥原理

“至少满足一个条件”对应集合的并集，非空子集奇加偶减：

$$\left|\bigcup_{i=1}^{k}A_i\right|=\sum_{\emptyset\ne S\subseteq\{1,\dots,k\}}(-1)^{|S|+1}\left|\bigcap_{i\in S}A_i\right|.$$

“一个坏条件都不满足”则用全集减去并集；也可把空集项（全集大小）纳入求和，符号改为 $(-1)^{|S|}$。取模减法须规范到非负余数。

例：统计 [1,n] 中至少被一个 a[i] 整除的整数。交集大小是 $\lfloor n/\operatorname{lcm}(a_i:i\in S)\rfloor$；只有两两互质时才能把 lcm 换成乘积。

下面使用 lcm，支持重复或不互质的除数。要求 0 <= n <= 10^18、a[i] >= 1、k <= 20；复杂度 O(k 2^k log n)，答案是整数，不取模。

```
ll countMultiples(ll n, const vll& a) {
    int k = sz(a);
    __int128 ans = 0;
    for (int mask = 1; mask < (1 << k); ++mask) {
        ll l = 1;
        bool over = false;
        for (int i = 0; i < k; ++i) {
            if(!(mask >> i & 1)) continue;
            ll t = a[i] / gcd(l, a[i]);
            if(l > n / t) { over = true; break; }
            l *= t;
        }
        if(over) continue;
        if(__builtin_popcount((unsigned)mask) & 1) ans += n / l;
        else ans -= n / l;
    }
    return (ll)ans;
}
```

递推计数与错排

先定义状态，按最后一步或一个指定元素的去向分类；各类必须互不重叠且覆盖全部方案。空对象通常贡献 1 种方案，例如 dp[0] = 1。

错排 D_n：n 个不同元素的排列中，每个元素都不在原位置。

$$D_0 = 1, \quad D_1 = 0, \quad D_n = (n-1)(D_{n-1} + D_{n-2}) \quad (n \geq 2).$$

推导：元素 1 选一个非原位 j，共 n-1 种；若 j 回到位置 1，剩下 D_{n-2} 种，否则压缩成 D_{n-1} 种。

容斥表达式：$D_n = \sum_{i=0}^{n} (-1)^i \binom{n}{i} (n-i)!$。恰好有 r 个元素在原位的排列数为 $\binom{n}{r} D_{n-r}$（0 <= r <= n）。

```
vll derange(int n) { // n >= 0, O(n)
    vll d(n + 1);
    d[0] = 1;
    for (int i = 2; i <= n; ++i)
        d[i] = (i - 1LL) * ((d[i - 1] + d[i - 2]) % MOD) % MOD;
    return d;
}
```

Catalan 数（卡特兰数）

Cat_n 统计 n 对合法括号；也统计从 (0,0) 到 (n,n)、只向右或向上且始终满足 y <= x 的路径，以及 n+1 个叶子的有序满二叉树。

$$\mathrm{Cat}_0 = 1, \qquad \mathrm{Cat}_n = \sum_{i=0}^{n-1} \mathrm{Cat}_i \mathrm{Cat}_{n-1-i} = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}.$$

括号递推按第一个左括号匹配的位置拆成 (A)B；组合数差式可由反射法得到。前几项（从 n=0 开始）：1, 1, 2, 5, 14, 42。

下面采用差式，避免额外除以 n+1；仍要求先 initComb(2*n, p)* 且 2n < p。题目给括号总长度 L 时，L 为奇数答案为 0，否则计算 catalan(L/2)。

```
ll catalan(int n) { // n >= 0, 依赖 C; 每次 O(1)
    return (C(2 * n, n) - C(2 * n, n - 1) + combMod) % combMod;
}
```

组合 DP：硬币计数与循环顺序

硬币面值为互不相同的正整数 c[i]，目标和 s >= 0；同面值硬币不可区分。状态 dp[j] 表示组成和 j 的方案数，dp[0] = 1。以下时间复杂度 O(n s)、空间 O(s)。

选择规则	循环顺序	计数含义
每种无限次，不计顺序	外层硬币，内层金额递增	每个面值的使用次数决定方案
每个物品至多一次，不计顺序	外层物品，内层金额递减	按物品身份选择子集；相同面值但身份不同仍是不同物品
每种无限次，计顺序	外层金额递增，内层硬币	按最后一枚硬币分类，统计序列

```
ll coinWays(const vi& c, int s) { // 无限次，不计顺序
    vll dp(s + 1);
    dp[0] = 1;
    for (int v : c)
        for (int j = v; j <= s; ++j)
            dp[j] = (dp[j] + dp[j - v]) % MOD;
    return dp[s];
}
ll coinSeq(const vi& c, int s) { // 无限次，计顺序
    vll dp(s + 1);
    dp[0] = 1;
    for (int j = 1; j <= s; ++j)
        for (int v : c)
            if(v <= j) dp[j] = (dp[j] + dp[j - v]) % MOD;
    return dp[s];
}
```

例如 c = {1,2}、s = 3：不计顺序为 2 种（1+1+1、1+2），计顺序为 3 种（另有 2+1）。

生成函数基础

普通生成函数 $F(x)=\sum_{s\ge 0} f_s x^s$，记 $[x^s]F(x)=f_s$。指数记录大小或总和，系数记录方案数；x 是形式变量，不需要讨论数值收敛。

独立选择的生成函数相乘，总和对应指数相加。设 $A(x)=\sum_i a_i x^i$、$B(x)=\sum_j b_j x^j$，则

$$[x^s]A(x)B(x)=\sum_{i=0}^{s} a_i b_{s-i}.$$

这就是卷积，也就是合并两组选择的计数 DP。

单个变量 / 一类物品的选择	生成函数
大小 v 的物品选 0 或 1 个	$1+x^v$
大小 v 的物品可选任意个	$1+x^v+x^{2v}+\cdots=1/(1-x^v)$
大小 v 的物品最多选 b 个	$1+x^v+\cdots+x^{bv}=(1-x^{(b+1)v})/(1-x^v)$
整数变量满足 l <= y <= u	$x^l+x^{l+1}+\cdots+x^u$ （0 <= l <= u）

不计顺序的无限硬币计数：

$$\text{coinWays}(c,s)=[x^s]\prod_i \frac{1}{1-x^{c_i}}.$$

计顺序时，先令 $A(x)=\sum_i x^{c_i}$，长度为 t 的序列对应 $A(x)^t$，包括空序列后：

$$\text{coinSeq}(c,s)=[x^s]\sum_{t\ge 0} A(x)^t=[x^s]\frac{1}{1-A(x)}.$$

插板法对应 $[x^m](1-x)^{-k}=\binom{m+k-1}{k-1}$；统一上界 b 时乘上 $(1-x^{b+1})^k$，展开分子就得到上面的容斥公式。

Catalan 递推对应 $F(x)=1+xF(x)^2$。基础练习先用 DP 求系数，只保留到目标次数；本节不依赖 FFT / NTT，也不包含形式幂级数求逆、求对数等进阶算法。

备用：线性代数

矩阵快速幂

保持固定维度的写法，按题修改 K。矩阵元素应先规范到 [0, MOD)，MOD 取 998244353 或 1e9 + 7。

```
const int K = 2;
struct Mat {
    ll a[K][K] = {};
    Mat operator*(const Mat& b) const {
        Mat r;
        for (int i = 0; i < K; ++i) {
            for (int k = 0; k < K; ++k) {
                if(!a[i][k]) continue;
                for (int j = 0; j < K; ++j)
                    r.a[i][j] = (r.a[i][j] + a[i][k] * b.a[k][j]) % MOD;
            }
        }
        return r;
    }
};
Mat qpow(Mat a, ll b) {
    Mat r;
    for (int i = 0; i < K; ++i) r.a[i][i] = 1;
    while(b) {
        if(b & 1) r = r * a;
```

```
        a = a * a;
        b >>= 1;
    }
    return r;
}
```

浮点高斯消元

n 行 n+1 列的增广矩阵，0 下标。会修改 A，唯一解写入 ans。返回 true 表示唯一解；false 表示没有唯一解。本版不区分无解和无穷多解。

```
bool gauss(vector<vector<db>>& A, vector<db>& ans) {
    int n = sz(A);
    for (int i = 0; i < n; ++i) {
        int p = i;
        for (int j = i + 1; j < n; ++j)
            if(fabs(A[j][i]) > fabs(A[p][i])) p = j;
        if(fabs(A[p][i]) < eps) return false;
        swap(A[i], A[p]);
        db v = A[i][i];
        for (int j = i; j <= n; ++j) A[i][j] /= v;
        for (int j = 0; j < n; ++j) {
            if(j == i) continue;
            db k = A[j][i];
            A[j][i] = 0;
            for (int t = i + 1; t <= n; ++t) A[j][t] -= k * A[i][t];
        }
    }
    ans.resize(n);
    for (int i = 0; i < n; ++i) ans[i] = A[i][n];
    return true;
}
```

模意义行列式

要求 MOD 为素数。0 下标，会修改 A；依赖前面的整数 qpow。

```
ll det(vector<vll>& A) {
    int n = sz(A);
    ll r = 1;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            A[i][j] %= MOD;
            if(A[i][j] < 0) A[i][j] += MOD;
        }
    }
    for (int i = 0; i < n; ++i) {
        int p = i;
        while(p < n && !A[p][i]) ++p;
        if(p == n) return 0;
        if(p != i) { swap(A[i], A[p]); r = (MOD - r) % MOD; }
        r = r * A[i][i] % MOD;
        ll iv = qpow(A[i][i], MOD - 2);
        for (int j = i + 1; j < n; ++j) {
            ll k = A[j][i] * iv % MOD;
            for (int t = i; t < n; ++t)
                A[j][t] = (A[j][t] - k * A[i][t] % MOD + MOD) % MOD;
        }
    }
    return r;
}
```

备用：数值积分与格点计数

辛普森积分

替换 f(x)，确认区间无奇点。调用 asr(l, r, 1e-8)；若达到递归深度上限，返回当前近似值，不保证误差达标。

```
db a, b, c, d;
db f(db x) { return (c * x + d) / (a * x + b); }
db simpson(db l, db r) {
    db m = (l + r) / 2;
    return (f(l) + 4 * f(m) + f(r)) * (r - l) / 6;
}
db asr(db l, db r, db e, db s, int dep) {
    db m = (l + r) / 2;
    if(m == l || m == r) return s;
    db x = simpson(l, m), y = simpson(m, r), delta = x + y - s;
    if(fabs(delta) <= 15 * e || dep == 0) return x + y + delta / 15;
    return asr(l, m, e / 2, x, dep - 1) + asr(m, r, e / 2, y, dep - 1);
}
db asr(db l, db r, db e) { return asr(l, r, e, simpson(l, r), 30); }
```

ewpage

计算几何

Every section builds on 基础. Copy top to bottom; a section only uses the ones above it.

Conventions: - points are passed by value (16 bytes, cheaper to type than `const P&`) - `cross(o, a, b) > 0` means o -> a -> b turns counter-clockwise (b is left of oa) - polygons are always counter-clockwise (CCW) - integer coordinates: prefer exact math, `using db = long long;` and `sgn(x) = (x > 0) - (x < 0)`. cross products reach 4e18 when coords are 1e9, so watch overflow - output: `cout << fixed << setprecision(10) << ans << "\n";`

基础

```
using db = double;
const db eps = 1e-9, PI = acos(-1.0);
int sgn(db x) { return fabs(x) < eps ? 0 : (x < 0 ? -1 : 1); }
struct P {
    db x, y;
    P(db x = 0, db y = 0) : x(x), y(y) {}
    P operator+(P b) const { return P(x + b.x, y + b.y); }
    P operator-(P b) const { return P(x - b.x, y - b.y); }
    P operator*(db k) const { return P(x * k, y * k); }
    P operator/(db k) const { return P(x / k, y / k); }
    bool operator<(P b) const { return sgn(x - b.x) ? x < b.x : sgn(y - b.y) < 0; }
    bool operator==(P b) const { return !sgn(x - b.x) && !sgn(y - b.y); }
};
istream& operator>>(istream& in, P& p) { return in >> p.x >> p.y; }
db dot(P a, P b) { return a.x * b.x + a.y * b.y; }
db det(P a, P b) { return a.x * b.y - a.y * b.x; }
db cross(P o, P a, P b) { return det(a - o, b - o); }
db len(P a) { return sqrt(dot(a, a)); }
P rot(P a, db t) { return P(a.x * cos(t) - a.y * sin(t), a.x * sin(t) + a.y * cos(t)); } // CCW by t
P rot90(P a) { return P(-a.y, a.x); }
```

点和线

A line or segment is given by two points `a, b`.

```
P proj(P p, P a, P b) { // projection of p onto line ab
    P d = b - a;
    return a + d * (dot(p - a, d) / dot(d, d));
}
```

```
P refl(P p, P a, P b) { return proj(p, a, b) * 2 - p; } // mirror of p over line ab
db disLine(P p, P a, P b) { return fabs(cross(a, b, p)) / len(b - a); }
db disSeg(P p, P a, P b) {
    if(sgn(dot(p - a, b - a)) < 0) return len(p - a);
    if(sgn(dot(p - b, a - b)) < 0) return len(p - b);
    return disLine(p, a, b);
}
bool onSeg(P p, P a, P b) { // endpoints count
    return !sgn(cross(p, a, b)) && sgn(dot(a - p, b - p)) <= 0;
}
bool segInter(P a, P b, P c, P d) { // touching and collinear overlap count
    int d1 = sgn(cross(a, b, c)), d2 = sgn(cross(a, b, d));
    int d3 = sgn(cross(c, d, a)), d4 = sgn(cross(c, d, b));
    if(d1 * d2 < 0 && d3 * d4 < 0) return true;
    return onSeg(c, a, b) || onSeg(d, a, b) || onSeg(a, c, d) || onSeg(b, c, d);
}
P lineInter(P a, P b, P c, P d) { // line ab meets line cd; make sure they're not parallel
    db s1 = cross(c, d, a), s2 = cross(c, d, b);
    return (a * s2 - b * s1) / (s2 - s1);
}
vector<P> segSeg(P a, P b, P c, P d) { // segment intersection points; collinear overlap gives both ends
    if(sgn(det(b - a, d - c))) {
        if(!segInter(a, b, c, d)) return {};
        return {lineInter(a, b, c, d)};
    }
    vector<P> r;
    if(onSeg(a, c, d)) r.pb(a);
    if(onSeg(b, c, d)) r.pb(b);
    if(onSeg(c, a, b)) r.pb(c);
    if(onSeg(d, a, b)) r.pb(d);
    sort(all(r));
    r.erase(unique(all(r)), r.end());
    return r;
}
```

Polar sort: CCW starting from the positive x-axis. No `atan2`, so it's exact on integers.

```
int half(P a) { return sgn(a.y) < 0 || (!sgn(a.y) && sgn(a.x) < 0); } // [0, pi) -> 0, [pi, 2pi) -> 1
bool cmpAng(P a, P b) {
    int x = half(a), y = half(b);
    return x != y ? x < y : sgn(det(a, b)) > 0;
}
// sort(all(v), cmpAng); to sort around o, subtract o from every point first
```

多边形

```
db area(vector<P>& A) { // signed, CCW is positive
    db s = 0;
    int n = sz(A);
    for (int i = 0; i < n; ++i) s += det(A[i], A[(i + 1) % n]);
    return s / 2;
}
int inPoly(P p, vector<P>& A) { // any simple polygon: 2 on edge, 1 inside, 0 outside
    int n = sz(A), w = 0;
    for (int i = 0; i < n; ++i) {
        P a = A[i], b = A[(i + 1) % n];
        if(onSeg(p, a, b)) return 2;
        int k = sgn(cross(a, b, p)), d1 = sgn(a.y - p.y), d2 = sgn(b.y - p.y);
        if(k > 0 && d1 <= 0 && d2 > 0) ++w;
        if(k < 0 && d2 <= 0 && d1 > 0) --w;
    }
    return w != 0;
```

```
}
bool inConvex(P p, vector<P>& A) { // A CCW, no 3 collinear, n >= 3; edge counts as inside; O(log n)
    int n = sz(A);
    if(sgn(cross(A[0], A[1], p)) < 0 || sgn(cross(A[0], A[n - 1], p)) > 0) return false;
    int l = 1, r = n - 1;
    while(r - l > 1) {
        int m = (l + r) / 2;
        if(sgn(cross(A[0], A[m], p)) >= 0) l = m;
        else r = m;
    }
    return sgn(cross(A[l], A[r], p)) >= 0;
}
vector<P> convexCut(vector<P>& A, P a, P b) { // part of convex A left of line ab, still CCW
    vector<P> r;
    int n = sz(A);
    for (int i = 0; i < n; ++i) {
        P c = A[i], d = A[(i + 1) % n];
        int s1 = sgn(cross(a, b, c)), s2 = sgn(cross(a, b, d));
        if(s1 >= 0) r.pb(c);
        if(s1 * s2 < 0) r.pb(lineInter(a, b, c, d));
    }
    return r;
}
```

凸包

Andrew's monotone chain. Returns CCW, no collinear points, starting from the bottom-left point. To keep collinear points on the hull, change both `<= 0` to `< 0`.

```
vector<P> hull(vector<P> A) {
    sort(all(A));
    A.erase(unique(all(A)), A.end());
    int n = sz(A), k = 0;
    if(n <= 2) return A;
    vector<P> H(2 * n);
    for (int i = 0; i < n; ++i) { // lower hull
        while(k >= 2 && sgn(cross(H[k - 2], H[k - 1], A[i])) <= 0) --k;
        H[k++] = A[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; --i) { // upper hull
        while(k >= t && sgn(cross(H[k - 2], H[k - 1], A[i])) <= 0) --k;
        H[k++] = A[i];
    }
    H.resize(k - 1);
    return H;
}
```

旋转卡壳

Diameter (farthest pair). For each edge, the farthest point `j` only moves forward.

```
db diameter(vector<P> H) { // H from hull()
    int n = sz(H);
    if(n <= 1) return 0;
    db r = 0;
    for (int i = 0, j = 1; i < n; ++i) {
        P a = H[i], b = H[(i + 1) % n];
        while(cross(a, b, H[(j + 1) % n]) > cross(a, b, H[j])) j = (j + 1) % n;
        r = max({r, len(a - H[j]), len(b - H[j])});
    }
    return r;
}
```

Width (min distance between two parallel supporting lines): same loop, start `r = 1e18` and use `r = min(r, cross(a, b, H[j]) / len(b - a));`

Min-area bounding rectangle: one side lies on a hull edge. Besides the farthest point `j`, keep `k` (farthest along the edge) and `l` (farthest backwards). All three only move forward.

```
db minRect(vector<P> H) { // H from hull(), n >= 3
    int n = sz(H);
    db res = 1e18;
    for (int i = 0, j = 1, k = 1, l = 0; i < n; ++i) {
        P a = H[i], b = H[(i + 1) % n], e = b - a;
        while(cross(a, b, H[(j + 1) % n]) > cross(a, b, H[j])) j = (j + 1) % n;
        while(dot(e, H[(k + 1) % n] - H[k]) > 0) k = (k + 1) % n;
        if(i == 0) l = j;
        while(dot(e, H[(l + 1) % n] - H[l]) < 0) l = (l + 1) % n;
        db w = cross(a, b, H[j]) / len(e), h = dot(e, H[k] - H[l]) / len(e);
        res = min(res, w * h);
    }
    return res;
}
```

半平面交

Each directed line `L(a, b)` keeps its left side. O(n log n). Always add a big bounding box (4 more lines) first: the code assumes the answer is bounded.

```
struct L {
    P p, v;
    db ang;
    L() {}
    L(P a, P b) : p(a), v(b - a) { ang = atan2(v.y, v.x); }
    bool operator<(const L& b) const { return ang < b.ang; }
};
bool onRight(L l, P a) { return sgn(det(l.v, a - l.p)) < 0; }
P inter(L a, L b) { return lineInter(a.p, a.p + a.v, b.p, b.p + b.v); }
vector<P> halfPlane(vector<L> ls) { // CCW polygon; empty if no area
    sort(all(ls));
    int n = sz(ls), h = 0, t = 0;
    vector<L> q(n);
    vector<P> p(n);
    q[0] = ls[0];
    for (int i = 1; i < n; ++i) {
        while(h < t && onRight(ls[i], p[t - 1])) --t;
        while(h < t && onRight(ls[i], p[h])) ++h;
        if(!sgn(det(ls[i].v, q[t].v))) { // parallel to the last one
            if(dot(ls[i].v, q[t].v) < 0) return {}; // opposite: empty
            if(onRight(q[t], ls[i].p)) continue; // new one is looser
            q[t] = ls[i];
        }
        else q[++t] = ls[i];
        if(h < t) p[t - 1] = inter(q[t - 1], q[t]);
    }
    while(h < t && onRight(q[h], p[t - 1])) --t;
    while(h < t && onRight(q[t], p[h])) ++h;
    if(t - h <= 1) return {};
    p[t] = inter(q[t], q[h]);
    return vector<P>(p.begin() + h, p.begin() + t + 1);
}
```

闵可夫斯基和

{a + b}. Start both hulls at their lowest (then leftmost) point and merge the edges by angle.

```
void lowFirst(vector<P>& A) {
    int k = 0;
    for (int i = 1; i < sz(A); ++i)
        if(mp(A[i].y, A[i].x) < mp(A[k].y, A[k].x)) k = i;
    rotate(A.begin(), A.begin() + k, A.end());
}
vector<P> minkowski(vector<P> A, vector<P> B) { // A, B CCW, no collinear points
    lowFirst(A), lowFirst(B);
    int n = sz(A), m = sz(B), i = 0, j = 0;
    vector<P> C = {A[0] + B[0]};
    while(i < n || j < m) {
        P a = A[(i + 1) % n] - A[i % n], b = B[(j + 1) % m] - B[j % m];
        int c = sgn(det(a, b));
        if(j == m || (i < n && c > 0)) C.pb(C.back() + a), ++i;
        else if(i == n || c < 0) C.pb(C.back() + b), ++j;
        else C.pb(C.back() + a + b), ++i, ++j;
    }
    C.pop_back();
    return C;
}
```

Distance between two convex polygons: it equals the distance from the origin to A + (-B).

```
db convexDist(vector<P> A, vector<P> B) { // both CCW, no collinear, n >= 3; 0 if they overlap
    for (auto& p : B) p = p * -1;
    vector<P> C = minkowski(A, B);
    if(inConvex(P(), C)) return 0;
    db r = 1e18;
    for (int i = 0; i < sz(C); ++i) r = min(r, disSeg(P(), C[i], C[(i + 1) % sz(C)]));
    return r;
}
```

圆

三角形的心 & 最小圆覆盖

```
P circumCenter(P a, P b, P c) { // a, b, c not collinear
    b = b - a, c = c - a;
    db d = 2 * det(b, c);
    return a + P(c.y * dot(b, b) - b.y * dot(c, c), b.x * dot(c, c) - c.x * dot(b, b)) / d;
}
P inCenter(P a, P b, P c) { // weights are the opposite side lengths
    db x = len(b - c), y = len(c - a), z = len(a - b);
    return (a * x + b * y + c * z) / (x + y + z);
}
P orthoCenter(P a, P b, P c) { return a + b + c - circumCenter(a, b, c) * 2; } // H = A + B + C - 2O
pair<P, db> minCircle(vector<P> A) { // randomized incremental, expected O(n)
    shuffle(all(A), mt19937(114514));
    int n = sz(A);
    P o = A[0];
    db r = 0;
    for (int i = 1; i < n; ++i) if(sgn(len(A[i] - o) - r) > 0) {
        o = A[i], r = 0;
        for (int j = 0; j < i; ++j) if(sgn(len(A[j] - o) - r) > 0) {
            o = (A[i] + A[j]) / 2, r = len(A[i] - o);
            for (int k = 0; k < j; ++k) if(sgn(len(A[k] - o) - r) > 0)
                o = circumCenter(A[i], A[j], A[k]), r = len(A[i] - o);
        }
    }
    return {o, r};
}
```

交点 & 切线

```
vector<P> lineCircle(P a, P b, P o, db r) { // line ab meets circle (o, r)
    P p = proj(o, a, b), e = (b - a) / len(b - a);
    db d = len(p - o);
    if(sgn(d - r) > 0) return {};
    db h = sqrt(max((db)0, r * r - d * d));
    if(!sgn(h)) return {p};
    return {p - e * h, p + e * h};
}
vector<P> circleCircle(P o1, db r1, P o2, db r2) { // same center returns empty
    db d = len(o2 - o1);
    if(!sgn(d) || sgn(d - r1 - r2) > 0 || sgn(d - fabs(r1 - r2)) < 0) return {};
    db a = (r1 * r1 - r2 * r2 + d * d) / (2 * d), h = sqrt(max((db)0, r1 * r1 - a * a));
    P e = (o2 - o1) / d, p = o1 + e * a;
    if(!sgn(h)) return {p};
    return {p - rot90(e) * h, p + rot90(e) * h};
}
vector<P> tangent(P p, P o, db r) { // touch points of tangents from p
    db d = len(p - o);
    if(sgn(d - r) < 0) return {};
    if(!sgn(d - r)) return {p};
    db t = acos(r / d);
    P e = (p - o) / d * r;
    return {o + rot(e, -t), o + rot(e, t)};
}
vector<pair<P, P>> tangents(P o1, db r1, P o2, db r2) { // outer common tangents; pass -r2 for inner ones
    P d = o2 - o1;
    db dr = r1 - r2, d2 = dot(d, d), h2 = d2 - dr * dr;
    if(!sgn(d2) || sgn(h2) < 0) return {};
    vector<pair<P, P>> res; // (touch point on circle 1, touch point on circle 2)
    for (int s : {-1, 1}) {
        P v = (d * dr + rot90(d) * sqrt(max((db)0, h2)) * s) / d2; // unit normal of the tangent
        res.pb({o1 + v * r1, o2 + v * r2});
    }
    if(!sgn(h2)) res.pop_back();
    return res;
}
```

圆与多边形面积交

Split into triangles (center, A[i], A[i+1]) and add their signed areas.

```
db ang(P a, P b) { return atan2(det(a, b), dot(a, b)); } // signed angle from a to b
db triCircle(P p, P q, db r) { // circle at origin, signed area of triangle (0, p, q) inside it
    P d = q - p;
    if(!sgn(len(d))) return 0;
    db a = dot(d, p) / dot(d, d), b = (dot(p, p) - r * r) / dot(d, d), D = a * a - b;
    if(D <= 0) return ang(p, q) * r * r / 2;
    db s = max((db)0, -a - sqrt(D)), t = min((db)1, -a + sqrt(D));
    if(t < 0 || s >= 1) return ang(p, q) * r * r / 2;
    P u = p + d * s, v = p + d * t;
    return (ang(p, u) + ang(v, q)) * r * r / 2 + det(u, v) / 2;
}
db polyCircle(vector<P>& A, P o, db r) { // positive if A is CCW
    db s = 0;
    int n = sz(A);
    for (int i = 0; i < n; ++i) s += triCircle(A[i] - o, A[(i + 1) % n] - o, r);
    return s;
}
```

两圆面积交 & 圆的面积并

```
db circleInterArea(P o1, db r1, P o2, db r2) {
    db d = len(o2 - o1);
    if(sgn(d - r1 - r2) >= 0) return 0;
    if(sgn(d - fabs(r1 - r2)) <= 0) return PI * min(r1, r2) * min(r1, r2);
    db a1 = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d)); // half of the central angle
    db a2 = acos((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d));
    return r1 * r1 * (a1 - sin(2 * a1) / 2) + r2 * r2 * (a2 - sin(2 * a2) / 2);
}
```

Union: Green's theorem. Walk every arc not covered by another circle and add $\int(x\,dy - y\,dx) / 2$ along it. $O(n^2 \log n)$.

```
db circleUnion(vector<P> o, vector<db> r) {
    int n = sz(o);
    db res = 0;
    for (int i = 0; i < n; ++i) {
        vector<pair<db, db>> ev; // covered angle ranges on circle i
        bool inside = false;
        for (int j = 0; j < n; ++j) if(j != i) {
            db d = len(o[j] - o[i]);
            int c = sgn(d + r[i] - r[j]);
            if(c < 0 || (!c && (sgn(d) || j < i))) { inside = true; break; } // i inside j (equal circles: keep one)
            if(sgn(d - r[i] - r[j]) >= 0 || sgn(d + r[j] - r[i]) <= 0) continue; // apart, or j inside i
            db base = atan2(o[j].y - o[i].y, o[j].x - o[i].x);
            db t = acos((r[i] * r[i] + d * d - r[j] * r[j]) / (2 * r[i] * d));
            db a = base - t, b = base + t;
            if(a < -PI) ev.pb({a + 2 * PI, PI}), ev.pb({-PI, b});
            else if(b > PI) ev.pb({a, PI}), ev.pb({-PI, b - 2 * PI});
            else ev.pb({a, b});
        }
        if(inside) continue;
        auto add = [&](db a, db b) { // uncovered arc [a, b]
            res += (r[i] * r[i] * (b - a) + o[i].x * r[i] * (sin(b) - sin(a)) - o[i].y * r[i] * (cos(b) - cos(a))) / 2;
        };
        sort(all(ev));
        db cur = -PI;
        for (auto [a, b] : ev) {
            if(a > cur) add(cur, a);
            cur = max(cur, b);
        }
        if(cur < PI) add(cur, PI);
    }
    return res;
}
```

极角扫描

Max number of points a circle of radius r can cover. Some optimal circle has a point A[i] on its boundary. Put the center at angle θ from A[i]; then A[j] is covered for θ in an interval. Sweep over θ. $O(n^2 \log n)$.

```
int maxCover(vector<P>& A, db r) {
    int n = sz(A), res = 1;
    for (int i = 0; i < n; ++i) {
        vector<pair<db, int>> ev;
        for (int j = 0; j < n; ++j) {
            db d = len(A[j] - A[i]);
            if(j == i || sgn(d - 2 * r) > 0) continue;
            db t = acos(min((db)1, d / (2 * r))), a = atan2(A[j].y - A[i].y, A[j].x - A[i].x) - t;
            if(a < 0) a += 2 * PI;
            ev.pb({a, -1}), ev.pb({a + 2 * t, 1}); // -1 sorts first, so touching counts
            ev.pb({a + 2 * PI, -1}), ev.pb({a + 2 * t + 2 * PI, 1}); // copy for wrap-around
        }
        sort(all(ev));
```

```
        int cnt = 1;
        for (auto [x, e] : ev) cnt -= e, res = max(res, cnt);
    }
    return res;
}
```

The same idea (sort the other points by angle around A[i], sweep with two pointers) also gives: max points on one side of a line through A[i], counting triangles that contain the origin, etc.

平面最近点对

Sweep by x, keep a strip of width d in a set ordered by y. O(n log n).

```
db closest(vector<P> A) {
    sort(all(A));
    set<pair<db, int>> S;
    db d = 1e18;
    for (int i = 0, l = 0; i < sz(A); ++i) {
        while(A[i].x - A[l].x > d) S.erase({A[l].y, l}), ++l;
        for (auto it = S.lower_bound({A[i].y - d, -1}); it != S.end() && it->fi < A[i].y + d; ++it)
            d = min(d, len(A[i] - A[it->se]));
        S.insert({A[i].y, i});
    }
    return d;
}
```

公式

- Pick: lattice polygon `S = I + B/2 - 1` (I interior points, B boundary points). Lattice points on segment (x1,y1)-(x2,y2): `gcd(|x1-x2|, |y1-y2|) + 1`
- Heron: `S = sqrt(p(p-a)(p-b)(p-c))`, `p = (a+b+c)/2`
- Angle between vectors: `acos(dot(a, b) / len(a) / len(b))`; signed: `ang(a, b)`
- Circular segment area: `r^2 / 2 * (theta - sin(theta))`